2-TIER CLOUD-BASED PRODUCT MANAGEMENT SYSTEM USING TERRAFORM & .NET

---

1. Overview

This project demonstrates the automated deployment of a scalable **Product Management System** using a **2-tier architecture on AWS**, provisioned entirely through **Infrastructure as Code (IaC)** using Terraform.

The application layer is developed using **.NET 8 Web API**, and the database layer is managed using **Amazon RDS (MySQL)**.

---

2. Objective

- Automate infrastructure provisioning using Terraform
- Implement a **2-tier architecture (Application + Database)**
- Deploy a **.NET Web API** application on cloud
- Enable full CRUD operations for product data
- Ensure secure communication between layers

---

3. Architecture Overview

Flow:

User → EC2 (.NET API) → RDS (MySQL Database)

Components:

- **Application Layer** → Hosted on EC2 instance
- **Database Layer** → Managed MySQL database on RDS

---

4. Technologies Used

- **.NET 8 Web API** → Backend application framework
- **Entity Framework Core (EF Core)** → ORM
- **Pomelo MySQL Provider** → MySQL connectivity
- **Terraform** → Infrastructure provisioning
- **AWS EC2** → Application hosting
- **AWS RDS (MySQL)** → Managed database
- **Linux (Amazon Linux 2023)** → Server OS
- **REST API** → Client-server communication

LALITHA PRASAD

5. Infrastructure Provisioned Using Terraform

a. Network Layer

- Virtual Private Cloud (VPC)
- Public Subnet for EC2
- Private Subnet for RDS
- Route Tables and Internet Gateway

b. Compute Layer

- EC2 Instance for .NET application
- User Data script for automated setup

c. Database Layer

- RDS MySQL instance (productdb)
- Secure database configuration

---

6. Security Groups Configuration

Application Server (EC2)

Inbound Rules:

- SSH (22) → Admin access
- Custom TCP (5001) → .NET API access

Outbound Rules:

- Allow all traffic (to access RDS)

Database Server (RDS)

Inbound Rules:

- MySQL (3306) → Only from EC2 Security Group

Security Highlights:

- Database is **not publicly accessible**
- Access restricted to application layer

---

7. Security Best Practices Implemented

- Least privilege access model
- Database isolated in private subnet

LALITHA PRASAD

- Controlled inbound rules
- No direct internet access to database

---

8. System Design

Application Layer

- Developed using .NET 8 Web API
- Handles API requests (CRUD operations)
- Uses EF Core for DB interaction

Database Layer

- MySQL database hosted in RDS
- Stores product information securely

---

9. Implementation Steps

1. Defined infrastructure using Terraform
2. Created VPC, subnets, and networking resources
3. Configured EC2 instance with user_data script
4. Installed .NET SDK automatically
5. Created Web API project dynamically
6. Connected application to RDS MySQL
7. Deployed and ran application
8. Tested APIs using curl/browser

---

10. Application Execution (Automated via Terraform)

Environment Setup (Auto in user_data)

- Install packages (dnf install)
- Install .NET SDK
- Configure environment variables

Application Setup

- Create project:

```
dotnet new webapi -n ProductInventory
```

LALITHA PRASAD

- Add dependencies:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Pomelo.EntityFrameworkCore.MySql
```

Run Application

```
dotnet run --urls http://0.0.0.0:5001
```

Access Application

http://<EC2-PUBLIC-IP>:5001/api/products

---

11. Application Features

- Add new products
- View all products
- View product by ID
- Update product
- Delete product

---

12. API Endpoints

Method	Endpoint	Description
GET	/api/products	Get all products
GET	/api/products/{id}	Get product
POST	/api/products	Add product
PUT	/api/products/{id}	Update product
DELETE	/api/products/{id}	Delete product

---

13. Challenges Faced

- User data script failures during provisioning
- Package compatibility issues (Amazon Linux 2023)
- .NET environment configuration issues
- Debugging cloud-init logs

---

14. Solutions Implemented

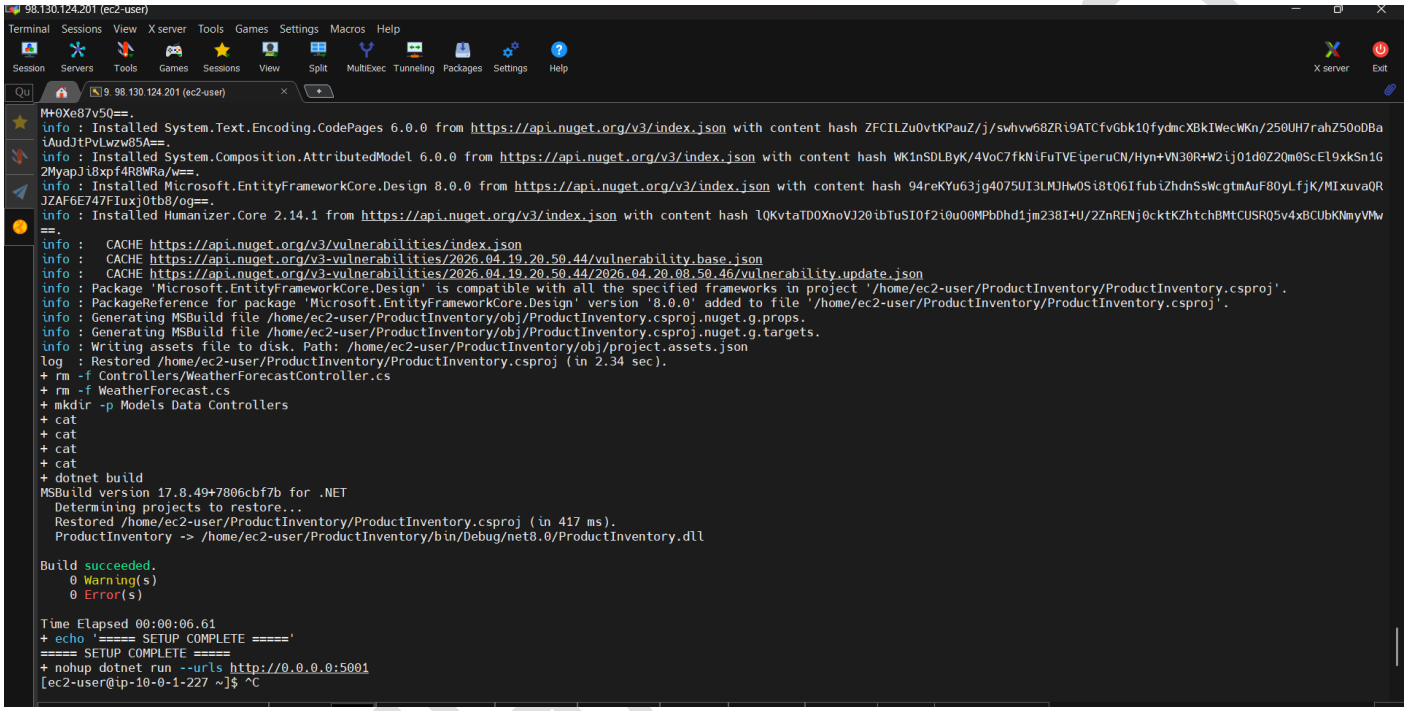
LALITHA PRASAD

- Used templatefile() for user_data
- Fixed package installation issues
- Set HOME environment for .NET
- Added logging (setup.log) for debugging

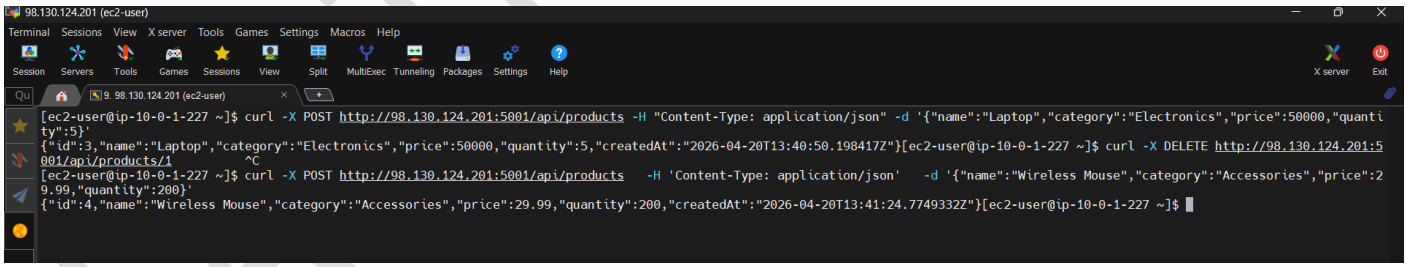
---

15. Outcome

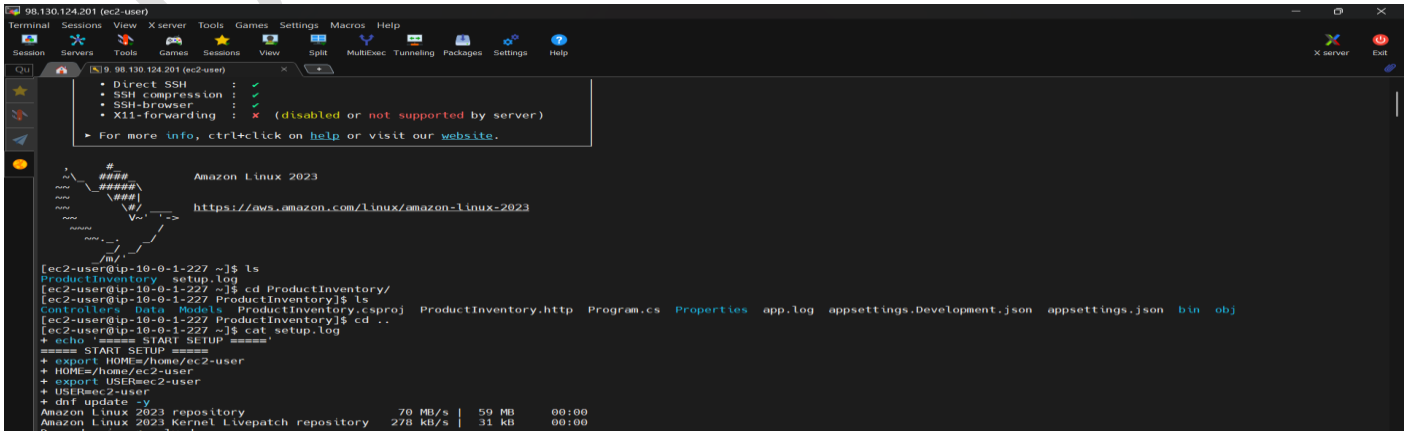
- Successfully deployed infrastructure using Terraform



- Fully automated application deployment



- Secure 2-tier architecture achieved



- Application accessible via public endpoint

Not secure 98.130.124.201:5001/api/products

retty-print


```
{
  "id": 1,
  "name": "Laptop",
  "category": "Electronics",
  "price": 50000,
  "quantity": 5,
  "createdAt": "2026-04-20T13:21:11.144916"
},
{
  "id": 2,
  "name": "Wireless Mouse",
  "category": "Accessories",
  "price": 29.99,
  "quantity": 200,
  "createdAt": "2026-04-20T13:24:30.141584"
}
```


---

16. Key Advantages of Terraform-Based Deployment

- Infrastructure automation
- Repeatable deployments
- Version-controlled infrastructure
- Reduced manual errors
- Faster provisioning

---

17. Conclusion

This project demonstrates real-world **DevOps and Cloud Engineering practices** by combining:

- Infrastructure as Code (Terraform)
- Cloud architecture (AWS 2-tier)
- Backend development (.NET API)

It provides a strong foundation for:

- Scalable applications
- CI/CD pipelines
- Containerized deployments

LALITHA P